

Reti (Computer Networks)

Introduzione al Laboratorio 2 Katarà

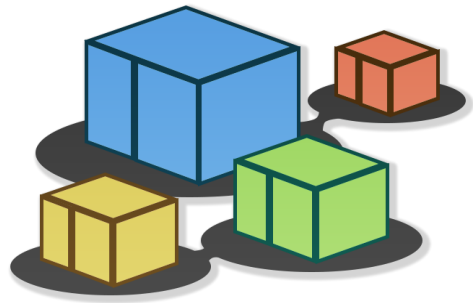
Docente: Paolo Casari

TA: Andrea Rosani

AA 2024/2025

Indice

- ❑ Katarà
- ❑ Docker
- ❑ Componenti fondamentali di Katarà
- ❑ Come usare Katarà
- ❑ Esempi di utilizzo

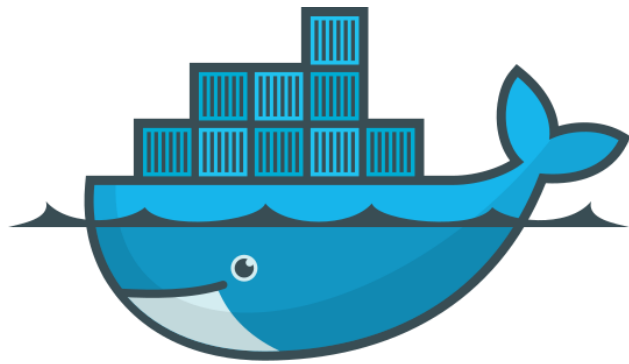


Kathará

Katarà

- ❑ Framework basato su Docker
- ❑ Permette di creare reti virtuali per sperimentare protocolli di rete
- ❑ Altamente modulare grazie a Docker images
- ❑ Controllo mirato di tutti i component esperimento





docker

Docker

- ❑ Progetto open-source che automatizza il processo di deployment delle applicazioni all'interno di contenitori software
- ❑ Gestisce "container" che eseguono processi in ambienti isolati, grazie all'isolamento delle risorse del kernel Linux
 - ❖ Container indipendenti coesistono sulla stessa istanza di Linux
- ❑ Evita l'installazione e la manutenzione di una macchina virtuale



Docker



- ❑ Un container, a differenza di una macchina virtuale, non include un sistema operativo separato
- ❑ Un container utilizza le funzionalità del kernel e sfrutta l'isolamento delle risorse (CPU, memoria, I/O a blocchi, rete) per isolare ciò che l'applicazione può vedere del sistema operativo
- ❑ Docker fornisce un ampio controllo ed ampia modularità:
 - ❖ Possiamo prendere un container, spostarlo su un altro sistema e funzionerà ancora
 - ❖ Permette la configurazione di rete (mappare porte input ed output, assegnare indirizzi, modificare le interfacce, ...)
- ❑ Strumento ideale per aziende che vogliono testare un prodotto prima di distribuirlo

Docker NON è...

❑ Cosa NON è Docker:

- ❖ Un ambiente di virtualizzazione
- ❖ Un kernel separato
- ❖ Un hypervisor

Virtual machine vs container

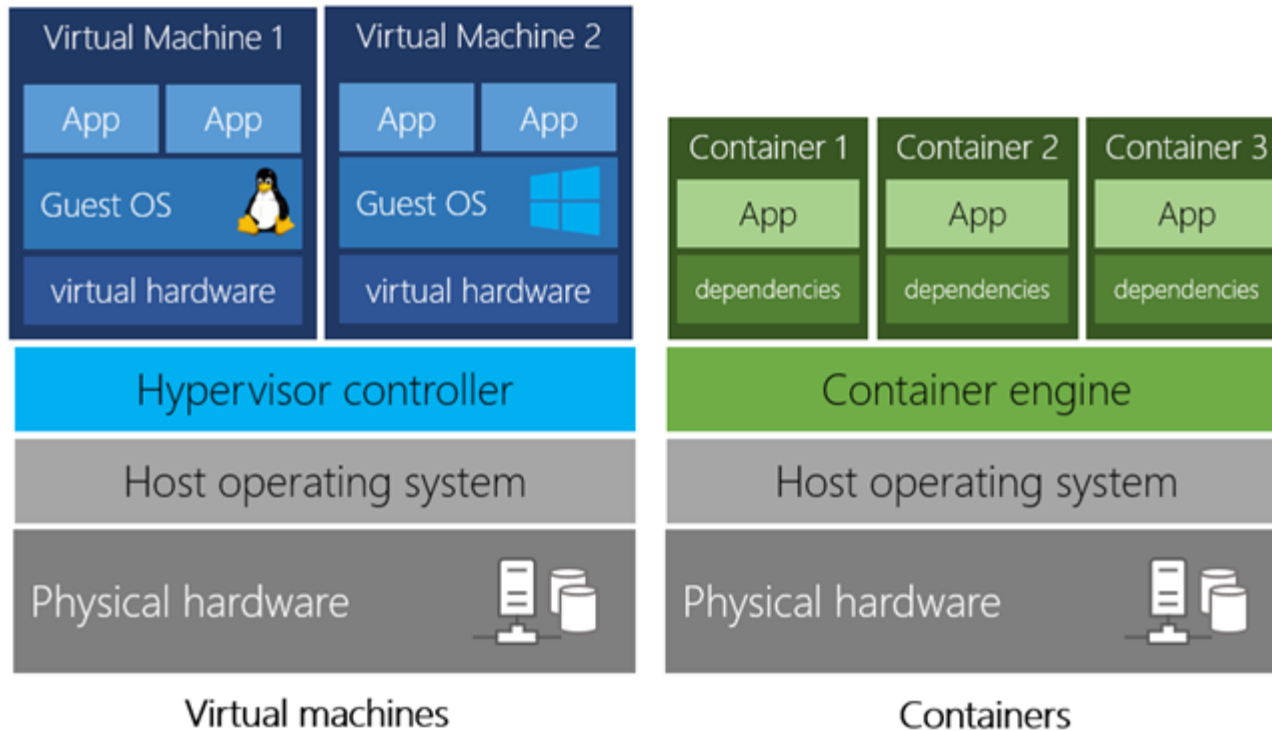


Immagine vs container Docker

Immagine

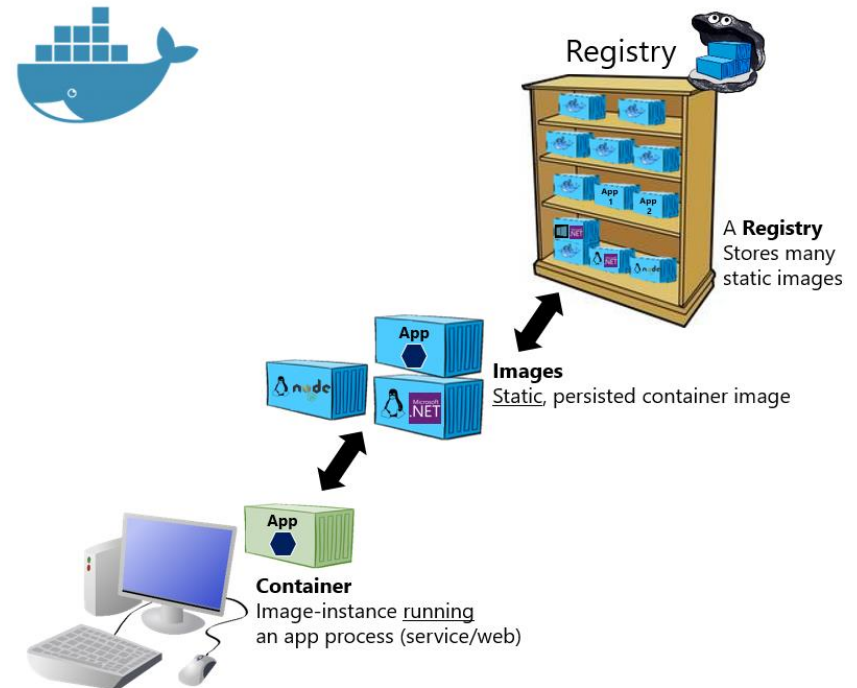
- ☐ Un modello immutabile per i container
- ☐ Può evolversi aggiungendo livelli all'immagine di partenza
- ☐ Può essere inserita in un registro, o da lì scaricata

Container

- ☐ Una istanza di una immagine
- ☐ Può essere avviata, fermata, riavviata
- ☐ Un container può essere ottenuto a partire da una immagine, personalizzata da un file di configurazione

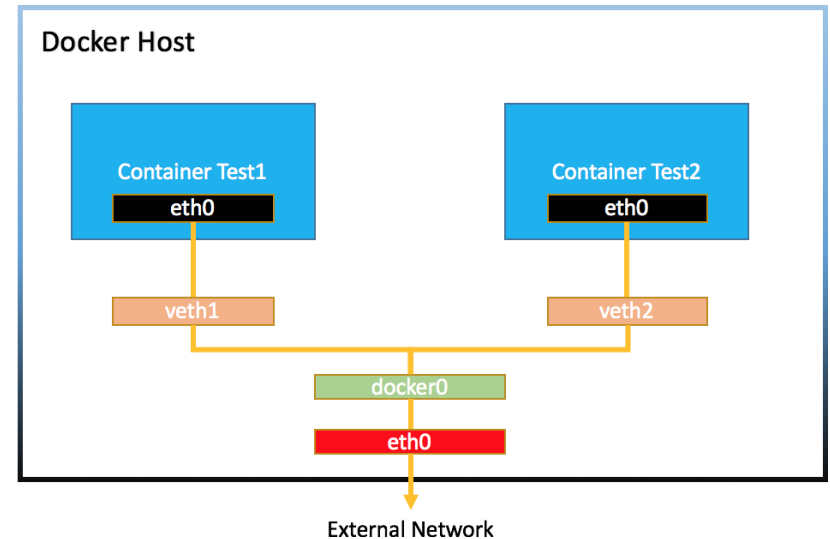
Docker Registry

- ❑ Archivio dove sono immagazzinate le immagini Docker



Altre funzionalità di Docker

- ❑ Modificare caratteristiche rete, mappare porte IO, indirizzi, interfacce
- ❑ Docker orchestrator: controllare più host su singola macchina
- ❑ Docker registry: è il github delle immagini Docker, permette di scaricare in locale ed eseguire immagini fatte da altri utenti



Installare Docker

- ❑ <https://docs.docker.com/get-docker/>
- ❑ Docker Server per Linux:
 - ❖ Ubuntu, Debian, CentOS, Fedora, RedHat, SUSE, Raspbian
- ❑ Docker Desktop:
 - ❖ Per MaC OS X, Windows 10, Windows 11

Docker Desktop per Windows

- ❑ <https://docs.docker.com/desktop/windows/install/>
- ❑ Occorre disporre di Windows 10 a partire dalla release 2004 (aprile 2020) o Windows 11
- ❑ Deve essere abilitato il WSL2
 - ❖ <https://docs.microsoft.com/en-us/windows/wsl/install-win10>
- ❑ Su Windows Pro può essere utilizzato Hyper-V
- ❑ I container sono condivisi tra tutti gli account utente
 - ❖ Perché tutti gli account usano la stessa VM per i container

Installare Docker Desktop su Windows

- ❑ <https://desktop.docker.com/win/stable/amd64/Docker%20Desktop%20Installer.exe>
- ❑ Durante l'installazione, flag su Linux containers
- ❑ ATTENZIONE: Se usate account diverso dall'amministratore, dovete aggiungere il vostro utente al Gruppo docker-users
 - ❖ Eseguire "Gestione Computer" come amministratore
 - ❖ Navigare a "Utenti e gruppi locali" > "Gruppi" > docker-users
 - ❖ Click destro > "Aggiungi al gruppo..." e aggiungete il vostro utente tra i membri
 - ❖ Le modifiche hanno effetto dopo un logout-login

- ❑ Dal greco Καθαρά
- ❑ Sistema di emulazione di rete basato su Docker containers
- ❑ Utile per testare e/o sviluppare protocolli di rete in un ambiente sandbox
- ❑ Altamente modulare grazie alle Docker images
- ❑ Controllo mirato di tutti i componenti all'interno dell'esperimento
 - ❖ Attraverso dei file di configurazione

Come Kathará usa Docker

- ❑ Docker images
 - ❖ Per definire gli applicativi
- ❑ Docker containers
 - ❖ Per eseguire le immagini in un ambiente ben definito
- ❑ Docker orchestrator
 - ❖ Per avere più containers in parallelo
- ❑ Docker File System
 - ❖ Per condividere file dal sistema ospitante al singolo container

Kathará su Linux

- ❑ <https://github.com/KatharaFramework/Kathara/wiki/Linux>
- ❑ Modalità di installazione diverse, in base alla distro. Per Ubuntu:
 - ❖ Aggiungere la chiave pubblica di Kathará al keyring:
 - `sudo apt-key adv --keyserver keyserver.ubuntu.com --recv-keys 21805A48E6CBBA6B991ABE76646193862B759810`
 - ❖ Aggiungere il repository di Kathará ai propri repos:
 - `sudo add-apt-repository ppa:katharaframework/kathara`
 - ❖ Aggiornare cache apt:
 - `sudo apt update`
 - ❖ Installare Kathará:
 - `apt install kathara`

Kathará su MacOS

- ❑ <https://github.com/KatharaFramework/Kathara/wiki/Macos>
- ❑ Viene supportata solo la versione x86
- ❑ Da <https://github.com/KatharaFramework/Kathara/releases> scegliere file .pkg, scaricarlo ed installarlo

- ❑ Prima di eseguire Kathará, Docker Desktop deve essere attivo
- ❑ Docker esegue su una macchina virtuale che esegue Linux ed è gestita da un hypervisor proprietario

Kathará su Windows

- ❑ <https://github.com/KatharaFramework/Kathara/wiki/Windows>
- ❑ Prerequisito: avere installato Docker Desktop
- ❑ Da <https://github.com/KatharaFramework/Kathara/releases> scegliere file .exe, scaricarlo ed installarlo

- ❑ Prima di eseguire Kathará, Docker Desktop deve essere attivo
- ❑ Kathará funziona solo su PowerShell !
 - ❖ Non è possibile utilizzare cmd

GESTIRE UNA SINGOLA MACCHINA

Avviare una singola macchina

- ❑ Questi comandi permettono di eseguire una singola macchina di Kathará (v command)

Comando	Azione
kathara vstart -n pc1	Avvia una macchina chiamata pc1
kathara vstart -n pc1 -i my-image --mem 128m	Avvia una macchina chiamata pc1 partendo da una immagine personalizzata con un limite di memoria
kathara vstart -n pc1 --eth 0:A	Avvia una macchina chiamata pc1 con una interfaccia di rete eth0 (non preconfigurata!)
kathara vclean -n pc1	Ferma la macchina chiamata pc1

- ❑ <https://www.kathara.org/man-pages/kathara.1.html>

File di configurazione

❑ Kathará dispone di un file di configurazione:

- ❖ ~/.config/kathara.conf
- ❖ C:\Users\<YOUR_USER_NAME>\.config\kathara.conf

```
{  
  "image": "kathara/quagga",  
  "manager_type": "docker",  
  "terminal": "Terminal",  
  "open_terminals": true,  
  "device_shell": "/bin/bash",  
  "net_prefix": "kathara",  
  "device_prefix": "kathara",  
  "debug_level": "INFO",  
  "print_startup_log": true,  
  "enable_ipv6": false,  
  "last_checked": 1667853085.048609,  
  "hosthome_mount": false,  
  "shared_mount": true,  
  "image_update_policy": "Prompt",  
  "shared_cd": false,  
  "remote_url": null,  
  "cert_path": null  
}
```

GESTIRE UN LABORATORIO

Gestire un laboratorio

- ❑ Laboratorio: insieme di file di configurazione che simulano un insieme di macchine
- ❑ Un laboratorio viene gestito con gli l-command
- ❑ File di configurazione necessari:
 - ❖ Lab descriptor: file *lab.conf*, descrive come è composto il laboratorio
 - ❖ Host descriptor: descrizione di un singolo host
Kathará cerca un file *<nome-host>.startup* per ogni file incluso nel lab descriptor

❖ <https://www.kathara.org/man-pages/kathara.1.html>

Gestire un laboratorio

Comando	Azione
kathara lstart	Avvia un laboratorio
kathara vconfig	Aggiungere/modificare l'interfaccia di rete su un singolo host, al volo
kathara ltest	Esegue un test sulle macchine di laboratorio (snapshot stato attuale laboratorio)
kathara lclean	Ferma un intero laboratorio
kathara lrestart	Riavvia un laboratorio già in esecuzione

Esercitazione

- ☐ Creare laboratorio base con pochi nodi
- ☐ Aggiungere un router per dividere il lab in due reti
- ☐ Modifica performance interfacce
- ☐ Analisi traffico tcp su rete

Esercitazione singola macchina

Riepilogo comandi:

kathara vstart -n pc1

Fa partire Katharà con una sola macchina chiamata pc1, si apre una shell dedicata dove è possibile digitare istruzioni linux

Provare ad esempio nella shell di pc1 a digitare il comando

ifconfig => (ip a)

Esso mostra le interfacce di rete, in particolare per pc1 si avrà soltanto l'interfaccia di loopback 127.0.0.1

Tornando alla shell del proprio pc digitare:

kathara vclean -n pc1

Il comando chiude l'istanza di pc1

Esercitazione multi macchina

Per testare il laboratorio è necessario creare (dove comodo) una directory che conterrà i files di configurazione.

```
mkdir tmp-kathara
```

```
cd tmp-kathara
```

⇒ Creare i file config del laboratorio

```
touch lab.conf
```

```
touch pc1.startup
```

```
touch pc2.startup
```

⇒ Editare i files con il contenuto slides seguente

```
vim lab.conf
```

(al posto di vim vanno bene anche emacs o altri editor solo testo)

```
vim pc1.startup
```

```
vim pc2.startup
```

⇒ Fare partire il laboratorio

```
kathara lstart
```

Esercitazione multi macchina

lab.conf

```
LAB_DESCRIPTION="laboratorio1"  
LAB_VERSION=1.0  
LAB_AUTHOR="Andrea"
```

```
pc1[0]="A"  
pc2[0]="A"
```

pc1.startup

```
ip a add 192.168.0.1/24 dev eth0  
ip link set eth0 up
```

pc2.startup

```
ip a add 192.168.0.2 dev/24 eth0  
ip link set eth0 up
```

Esercitazione multi macchina

Provare vari comandi nelle due macchine pc1 e pc2.

Testare il comando *ip*

Provare a fare ping da pc1 verso pc2 usando l'IP e viceversa

Nota: i tempi di ritardo sono differenti, Katharà usa il kernel

Ricordarsi di chiudere il laboratorio con

kathara lclean

FILE CONDIVISI TRA MACCHINE

Esercitazione file condivisi tra macchine

File condivisi tra tutte le macchine => cartella «Shared» creata da Katharà quando parte il laboratorio.

File su singola macchina soltanto => cartella con il nome pc, ad esempio pc1 o pc2. Il file si troverà nella root della macchina.

Fare qualche test

ESEMPIO CON UN ROUTER

Esempio con un router



Esempio con un router

lab.conf

```
LAB_DESCRIPTION="laboratorio1"  
LAB_VERSION=1.0  
LAB_AUTHOR="Andrea"
```

```
r1[0]="A"  
r1[1]="B"
```

```
pc1[0]="A"  
pc2[0]="B"
```

Esempio (2)

r1.startup

```
ip a add 192.168.0.100/24 dev eth0  
ip link set eth0 up  
ip a add 192.168.1.100/24 dev eth1  
ip link set eth1 up
```

pc1.startup

```
ip a add 192.168.0.1/24 dev eth0  
ip link set eth0 up
```

pc2.startup

```
ip a add 192.168.1.1/24 dev eth0  
ip link set eth0 up
```

Esercitazione con router

Avviare laboratorio.

Provare ping da pc1 a r1 e ping da pc2 a r1. Funzionano

Provare ping da pc1 a pc2. NON VA!

Perché? Manca il gateway!

⇒ Devo rifare le config inserendo il gw in ciascun file configurazione delle macchine.

In pc1.startup aggiungere la linea

```
route add default gw 192.168.0.100 dev eth0
```

In pc2.startup aggiungere la linea

```
route add default gw 192.168.1.100 dev eth0
```

Al riavvio di katara lstart lanciare comando route per vedere il gw.

Da pc1 raggiungo pc2, provare con un ping.

PROVE CON WIRESHARK

Esercitazione wireshark

lab.conf

```
pc1[0]=A
```

```
pc2[0]=A
```

```
wireshark[bridged]=true
```

```
wireshark[port]="3000:3000/tcp"
```

```
wireshark[image]="Iscrio/linuxserver/wireshark"
```

pc1.startup

```
ip address add 100.0.0.1/24 dev eth0
```

pc2.startup

```
ip address add 100.0.0.2/24 dev eth0
```


Esercitazione wireshark

<https://localhost:3000>

Collision domain

PROVE CON IPERF

File di configurazione

❑ Occorre modificare il file di configurazione:

❖ ~/.config/kathara.conf

❖ C:\Users\<YOUR_USER_NAME>\.config\kathara.conf

```
{  
  "image": "tiamilani/ubuntu_tc",  
  "manager_type": "docker",  
  "terminal": "Terminal",  
  "open_terminals": true,  
  "device_shell": "/bin/bash",  
  "net_prefix": "kathara",  
  "device_prefix": "kathara",  
  "debug_level": "INFO",  
  "print_startup_log": true,  
  "enable_ipv6": false,  
  "last_checked": 1667853085.048609,  
  "hosthome_mount": false,  
  "shared_mount": true,  
  "image_update_policy": "Prompt",  
  "shared_cd": false,  
  "remote_url": null,  
  "cert_path": null  
}
```

Esercitazione con iperf

Scaricare immagine tiamilani/ubuntu_tc => kathara.conf

iperf

Serve per misurare le performance.

Serve un server ed un client

Usare laboratorio con 2 macchine e router

su pc1 (server)

`iperf -s -i 1`

(listening, windows 128 default)

su pc2 (client)

`iperf -c 192.168.0.1 -t 20 -i 1`

(durata esperimento 20 secondi, ogni quanto intervallo vogliamo studiare la cosa -i 1)

Vedo quanti dati trasferiti e banda attiva

Posso modificare la TCP window (al momento è 4 MB => 10kB)

`iperf -c 192.168.0.1 -t 20 -i 1 -w 10k` su pc2

Vedo che la banda è scesa.

Guardo rtt medio 0.1 => w impostato a 300k, torno alle performances precedenti